大数据与人工智能
自动化学院

3.3-1导言

这次主要是学习

- Pytorch的数据读取机制DataLoader和Dataset的运行机制
- 然后学习图像的预处理模块transforms的原理
- 最后基于上面的所学实现人民币二分类的任务。

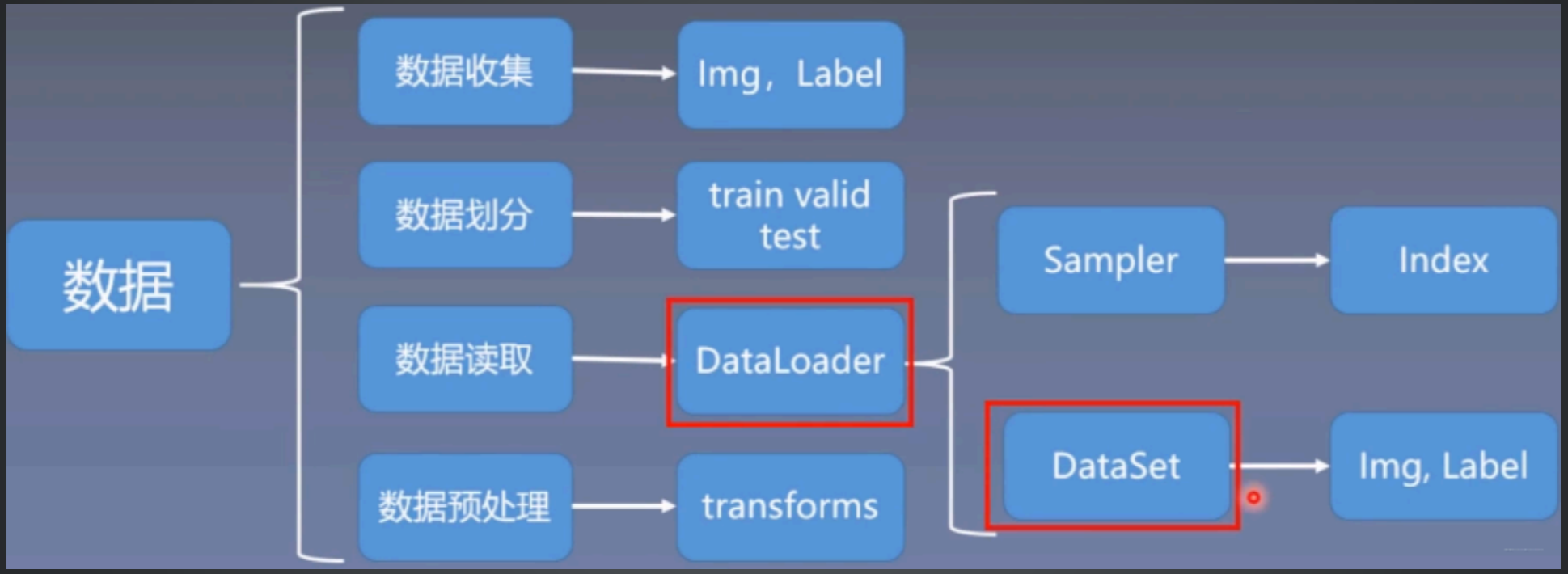
- 总结梳理

3.3-2. PYTORCH的数据读取机制

- 在学习Pytorch的数据读取之前，我们得先回顾一下这个数据读取到底是以什么样的逻辑存在的
- 上一节，我们已经整理了机器模型学习的五大模块，分别是**数据**，模型，损失函数，优化器，迭代训练：

而这里的数据读取机制，很显然是位于数据模块的一个小分支

下面看一下数据模块的详细内容：



- 数据模块中，又可以大致分为上面不同的子模块
- 今天学习的DataLoader和DataSet就是数据读取子模块中的核心机制。
- 下面正式开始DataLoader和Dataset的学习

3.3-2.1 DATALOADER

- `torch.utils.data.DataLoader()`: 构建可迭代的数据装载器
- 我们在训练的时候，每一个for循环，每一次iteration，就是从DataLoader中获取一个batch_size大小的数据的。

```
DataLoader( dataset,
            batch_size=1,
            shuffle=False,
            sampler=None,
            batch_sampler=None,
            num_workers=0,
            collate_fn=None,
            pin_memory=False,
            drop_last=False,
            timeout=0,
            worker_init_fn=None,
            multiprocessing_context=None)
```

DataLoader的参数很多，但我们常用的主要有5个：

- dataset: Dataset类， 决定数据从哪读取以及如何读取
- bathsize: 批大小
- num_works: 是否多进程读取机制
- shuffle: 每个epoch是否乱序
- drop_last: 当样本数不能被batchsize整除时， 是否舍弃最后一批数据

要理解这个drop_last， 首先，得先理解Epoch， Iteration和Batchsize的概念：

- Epoch： 所有训练样本都已输入到模型中，称为一个Epoch
- Iteration： 一次循环所需的样本输入到模型中，称为一个Iteration
- Batchsize： 一批样本的大小， 决定一个Epoch有多少个Iteration

举个例子就Ok了

- 假设样本总数80， Batchsize是8， 那么1Epoch=10 Iteration。
- 假设样本总数是87， Batchsize是8
    - 如果drop_last=True, 那么1Epoch=10Iteration
    - 如果等于drop_last=False， 那么1Epoch=11Iteration, 最后1个Iteration有7个样本。

提问：下面哪一个不是 `DataLoader` 的常用参数？

- A. `dataset`
- B. `batch_size`
- C. `optimizer`
- D. `shuffle`

正确答案: C

答案讲评: DataLoader 的常用参数包括 dataset（指定数据集）、batch_size（批次大小）、shuffle（是否打乱数据顺序）等。而 optimizer 是用于优化模型参数的，与数据读取无关

torch.utils.data.Dataset(): Dataset抽象类

- 所有自定义的Dataset都需要继承它，并且必须复写（自己根据需要重写）__getitem__()这个类方法。

```
class Dataset(object):

    def __getitem__(self, index):
        raise NotImplementedError

    def __add__(self, other):
        return ConcatDataset([self, other])
```

__getitem__方法是Dataset的核心

- 作用是接收一个索引，返回一个样本

看上面的函数，参数里面接收index，然后我们需要编写究竟如何根据这个索引去读取我们的数据部分。

提问：**在Pytorch的数据读取机制中，`DataLoader`的作用是什么？**

- A. 构建数据集
- B. 进行图像预处理
- C. 构建可迭代的数据装载器
- D. 训练模型

正确答案: C

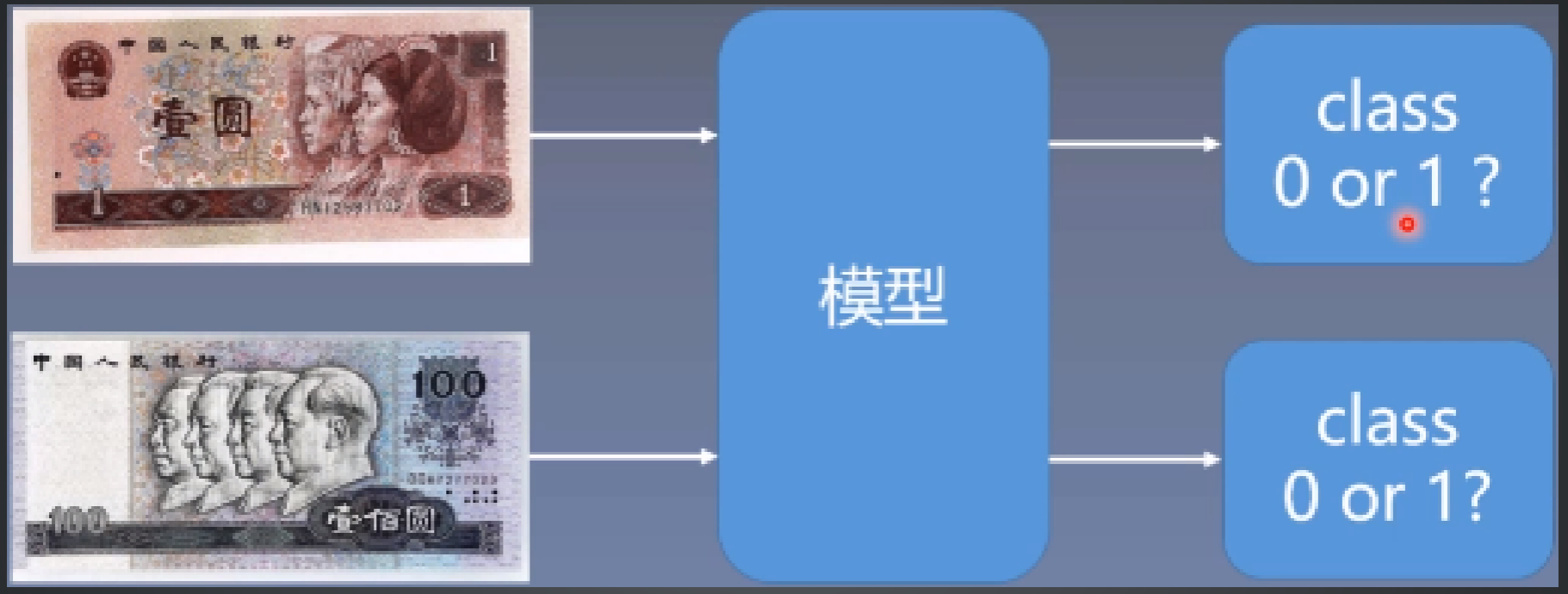
答案讲评: DataLoader 是 Pytorch 中用于构建可迭代数据装载器的类，它在每一次迭代时从数据集中提取一个批次的数据，并将它们传递给模型进行训练。它通过参数
如 batch_size、shuffle 和 num_workers 来控制数据的读取方式

3.3-2.3 数据读取机制具体怎么用呢?

- 上面只是介绍了两个数据读取机制用到的两个类，那么具体怎么用呢？ 我们就以人民币二分类的任务进行具体查看，
- 但是查看之前我们要带着关于数据读取的三个问题去看：

1. 读哪些数据？ 我们每一次迭代要去读取一个batch_size大小的样本，那么读哪些样本呢?
2. 从哪读数据？ 也就是在硬盘当中该怎么去找数据，在哪设置这个参数。
3. 怎么读数据?

下面我们从实验中边看边学习：人民币分类的任务其实也非常简单， 就是



- 我们的数据集是1块钱的图片100张，100块钱的图片100张
- 我们的任务就是训练一个模型，来帮助我们对这两类图片进行分类。

这个说清楚了之后， 我们下面就带着上面的三个问题，来看我们这个任务的数据读取部分。

数据读取的代码

不用具体看懂，看懂这里的逻辑就可以

```
#==========================================step 1/5 准备

# 数据的路径
split_dir = os.path.join('data', 'rmb_split')
train_dir = os.path.join(split_dir, 'train')
valid_dir = os.path.join(split_dir, 'valid')

## transforms模块，进行数据预处理
norm_mean = [0.485, 0.456, 0.406]
norm_std = [0.229, 0.224, 0.225]

train_transform = transforms.Compose([
    transforms.Resize((32, 32)),
```

- 首先一开始，是路径部分， 也就是训练集和测试集的位置，这个其实就是我们上面的第二个问题**从哪读数据**
- 然后是transforms图像数据的预处理部分， 这个不用管， 后面会介绍transforms这个模块
- 这次最重要的就是MyDataset实例，即**train_data和valid_data**还有后面的DataLoader，这个才是我们这次介绍的重点。

我们下面详细剖析（这个地方会涉及到代码的一些调试，所以尽量慢一些）：

- 我们从`train_data = RMBDataset(data_dir=train_dir, transform=train_transform)`开始
    - 这一句话里面的核心就是RMBDataset，这个是我们自己写的一个类，继承了上面的抽象类Dataset，并且重写了`__getitem__()`方法
    - RMBDataset这个类的目的就是传入数据的路径，和预处理部分（看参数），然后给我们返回数据
- 下面看它是怎么实现的(VS Code或者Pycharm里面按住控制键，然后点击这个类就进入具体实现)：

```python
class RMBDataset(Dataset):
    def __init__(self, data_dir, transform=None):
        """
        rmb面额分类任务的Dataset
        :param data_dir: str, 数据集所在路径
        :param transform: torch.transform，数据预处理
        """
        self.label_name = {"1": 0, "100": 1}
        self.data_info = self.get_img_info(data_dir)
        self.transform = transform

    def __getitem__(self, index):
        path_img, label = self.data_info[index]
```

- 这么多代码比较复杂，但是得养成读源码的习惯，依然是看逻辑关系
- 然后再具体深入进去看具体细节。
- 逻辑方面，这里面重点就是__getitem__()这个方法的实现了
    - 从这里面，我们要拿到我们的训练样本，
    - 那么怎么拿呢？ 这个函数的第一行，会看到有个data_info[index]， 我们只要给定了index， 那么就是通过这句代码进行获取样本的

- 所以上面的重点又落在了data_info[index]上面， 这句代码干了个什么事情呢?
- 那么就得看看它是咋来的，所以就该往上看这个类的初始化部分__init__
- 我们可以看到这个data_info是RMBDataset这个类的成员
- 我们会看到self.data_info = self.get_img_info(data_dir)这句代码， 就找到了data_info的来源，

- 我们又发现这个又调用了`get_img_info(data_dir)`方法， 这个才是最终的根源
- 所以我们又得看这个函数`get_img_info(data_dir)`做了什么?
    - 我们会发现这个函数的参数是data_dir, 也就是数据在的路径
    - 这个函数应该是要根据这个路径去找数据的
    - 这个函数写了这么多代码，其实就干了一件事，根据我们给定的路径去找数据，然后返回这个数据的位置和标签。

- get_img_info(data_dir)返回的是一个list， 而list的每个元素是元组，格式就是[(样本1_loc, label_1), (样本2_loc, label_2), ...(样本n_loc, label_n)]。
- 这个其实就是data_info拿到的一个list。
- 有了这个list，然后又给了data_info一个index，那么取数据不就很容易了吗？data_info[index] 不就取出了某个(样本i_loc, label_i)。

- 这样再回到__getitem__()这个方法， 就容易理解了
- 第一行我们拿到了一个样本的图片路径和标签。
- 第二行就是去找到图片，然后转成RGB数值。
- 第三行就是做了图片的数据预处理
- 最后返回了这张图片的张量形式和它的标签。

注意，这里是一个样本的张量形式和标签。 这就是RMBDataset这个类做的事情。

那么你可能有个疑问了，我们肯定不是要获取一张图片啊， 我们不是要获取batch_size张图片吗?

那么这个就要问下面的DataLoader了。

- 我们看这句代码`train_loader = DataLoader(dataset=train_data, batch_size=BATCH_SIZE, shuffle=True)`,
- 看DataLoader这个类，接收的参数就是上面的`RMBDataset`，我们知道这个是返回一个样本的张量和标签
- 然后又跟了一个`BATCH_SIZE`,

- 看到这个，你心里应该有数了，这个不就是说一个batch里面有多少个样本吗?
- 如果有了一个batch的样本数量，有了样本总数，就能得到总共有多少个batch了。
- 后面的shuffle，这个是说我取图片的时候，把顺序打乱一下

那么这个DataLoader在干啥事情呢？其实它在干这样的事情

- 我们只要指定了Batch_SIZE， 比如指定10个， 我们总共是有80个训练样本，那么就可以计算出批数是8
- 那么DataLoader就把样本分成8批顺序打乱的数据，每一个Batch_size里面有10个样本且都是张量和标签的形式

那么DataLoader是怎么做到的呢?

- 需要看DataLoader的源码
- 但是源码太长了，没法在这里具体显示
- 我们可以先看看这个train_loader到底是个啥：

```
<torch.utils.data.dataloader.DataLoader object at 0x000001D8C284DBC8>
```

看来这是一个DataLoader对象了， 也没法进行研究了

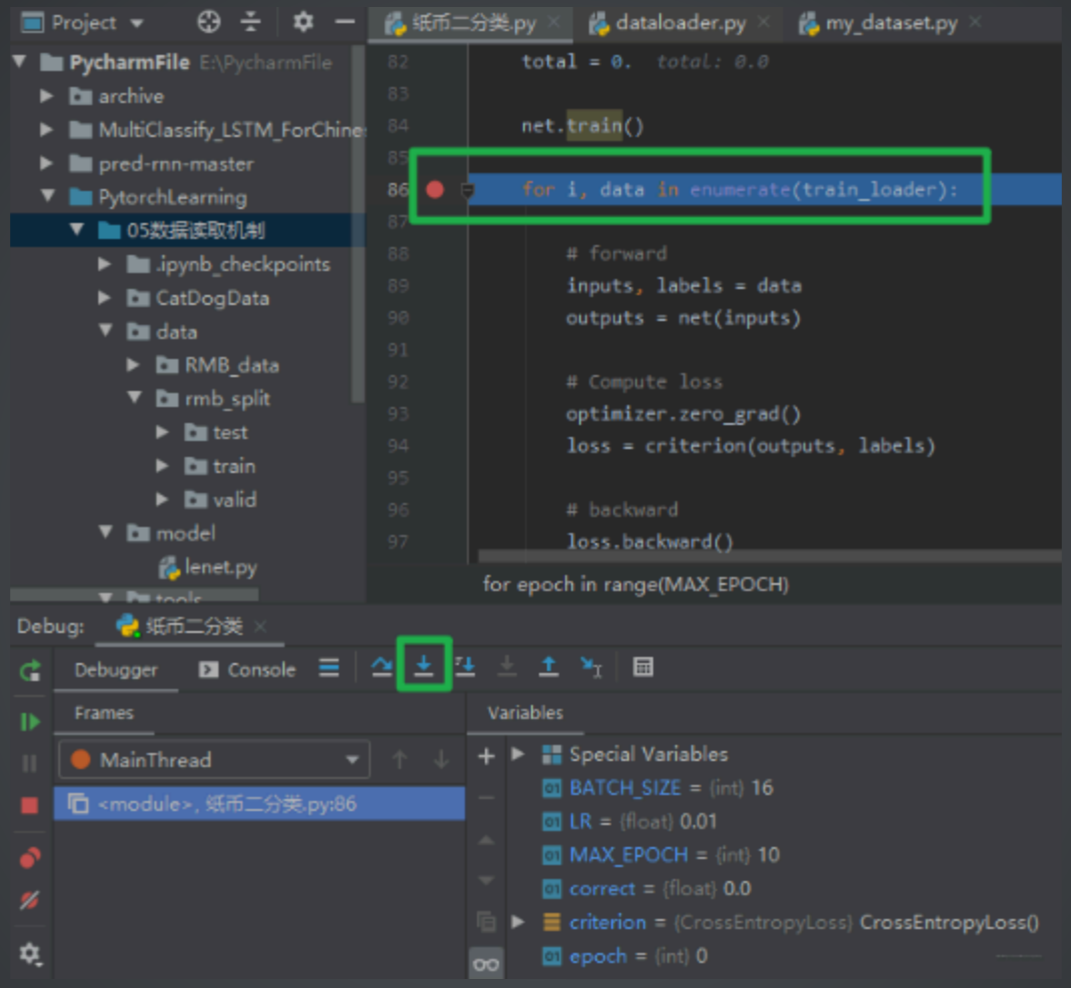
- 现在只知道这个东西能够返回那10批数据，每批里面有Batch_size个数据，赋值给了train_loader
- 显然这是一个可迭代的对象。
- 那么很容易就可以想到，如果下面我们具体训练的时候，肯定是要遍历这个train_loader
- 然后每一次取一批数据进行训练。

- 我们从具体使用的时候，看看每一批数据究竟是如何获得的?
- 下面我们就直接从训练的部分看，像中间的模型，损失函数，优化器不是重点，所以这里先不放上来:

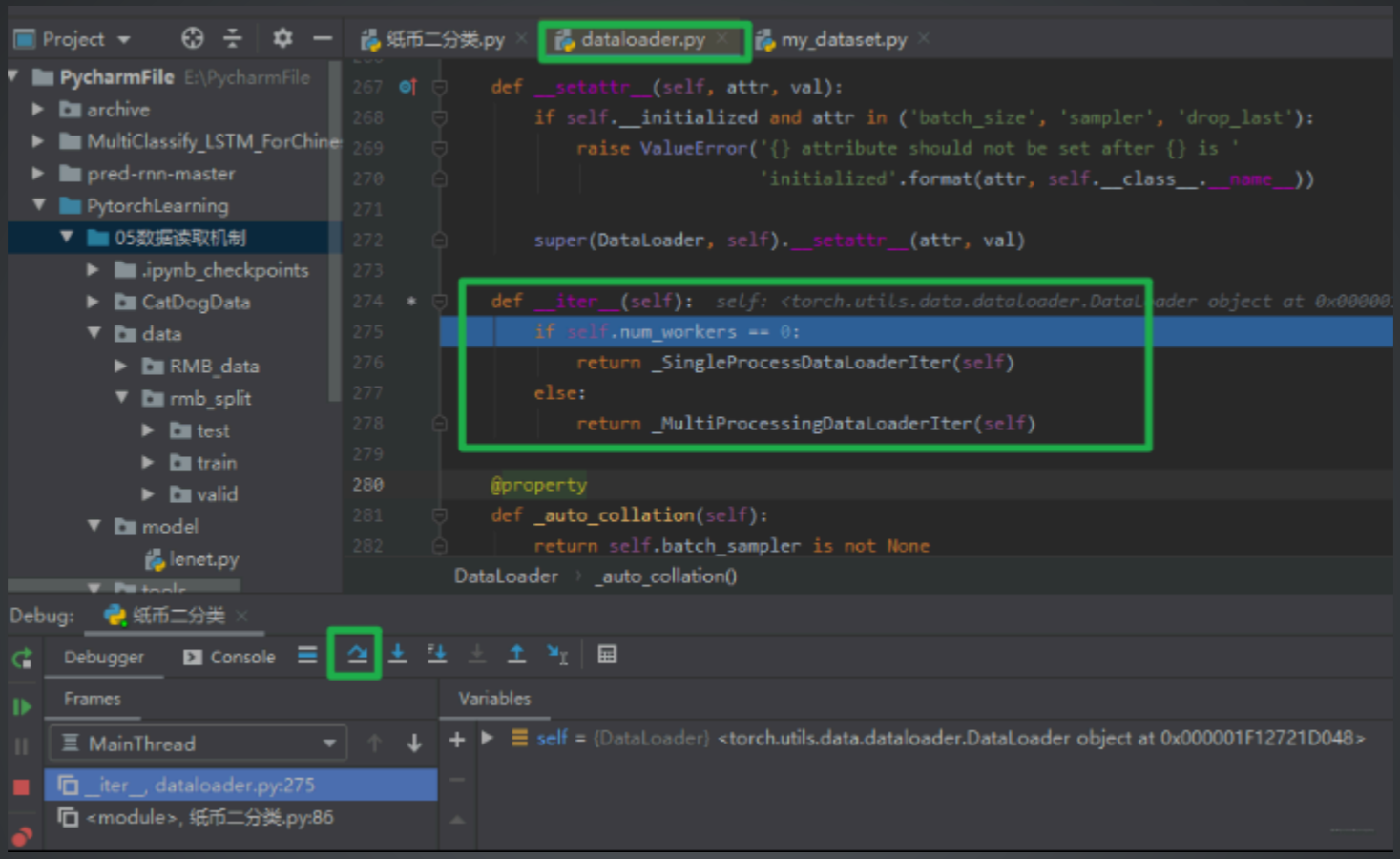
```
for epoch in range(MAX_EPOCH):
    loss_mean = 0.
    correct = 0.
    total = 0.

    net.train()

    for i, data in enumerate(train_loader):

        # forward
        inputs, labels = data
        outputs = net(inputs)
```

- 上面就是训练部分的核心了，这个比较好理解， 两层循环
- 外循环：表示的迭代Epoch，每一次Epoch+1也就是全部的训练样本喂入模型一次
- 每一个Epoch中，进行内循环：每一个i+1表示一个批次的训练样本喂入模型
- 那么数据读取具体使用的核心就是`for i, data in enumerate(train_loader)`这句话了

- 我们以调试的方式看看这个函数究竟是怎么去得到数据的?

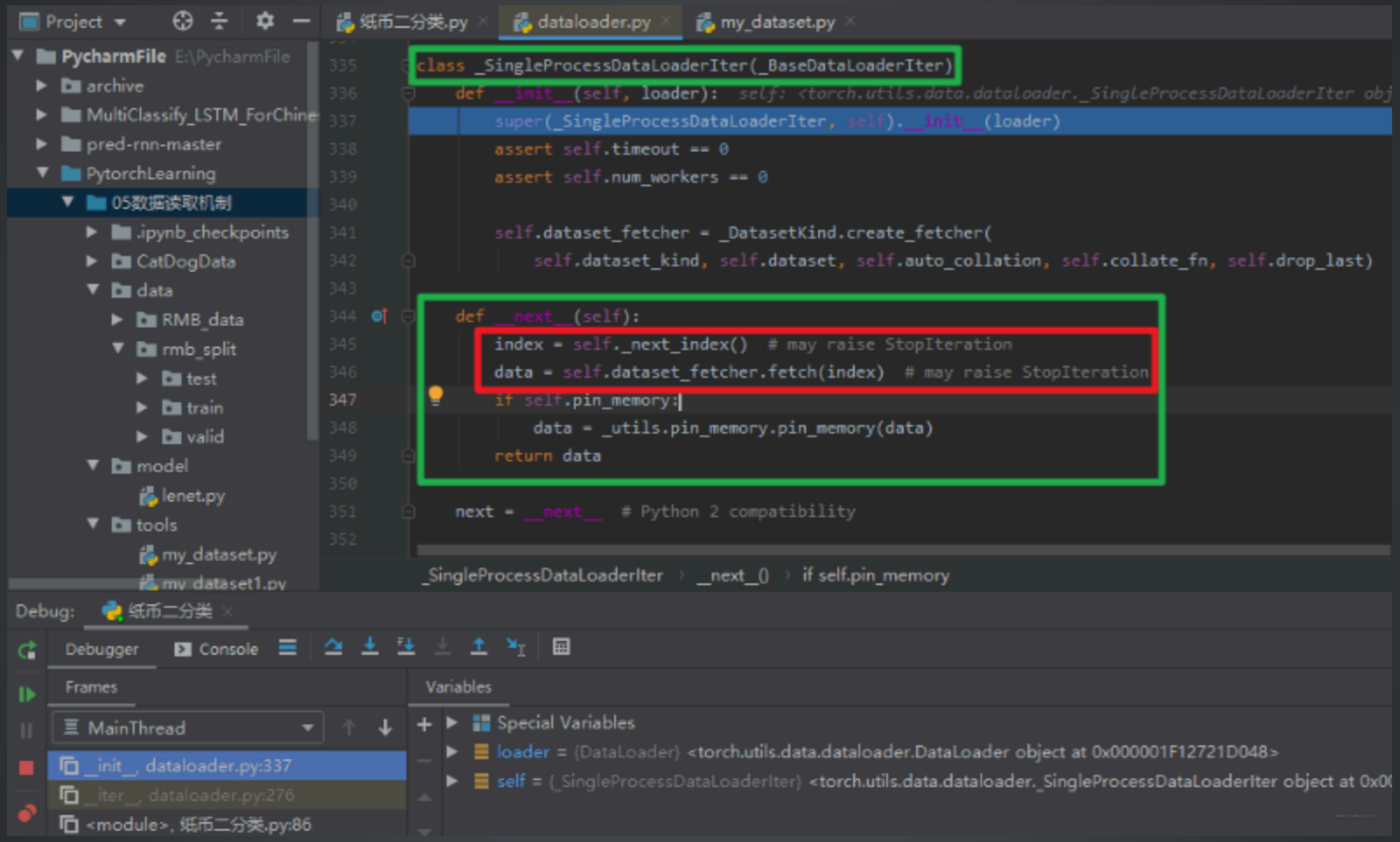


- 在这一行打断点，然后debug，程序运行到这一行，然后点击下面的stepinto步入这个函数里面
- 我们看看调用的DataLoader里面的哪个方法



- 这样就会看到，程序跳转到了DataLoader的__iter__(self)这个方法
- 简单的瞄一眼这个函数，就会发现就一个判断，说的啥呢？ 原来在说是用单进程还是用多进程读取机制进行处理
- 关于读取数据啥也没干。 所以这个也不是重点， 我们使用stepover进行下一步，

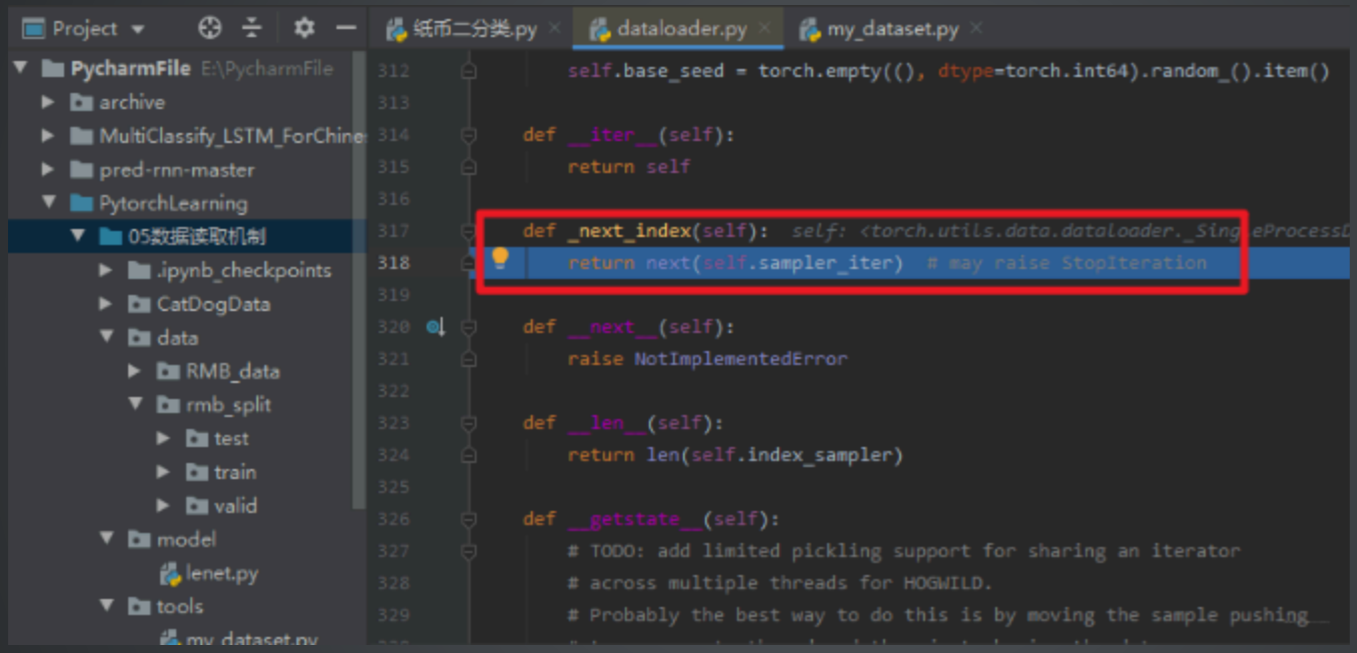
然后在stepinto进入单进程的这个机制里面



在这里面会看到点玄机了，这个机制里面比较重要的一个方法就是__next__(self),

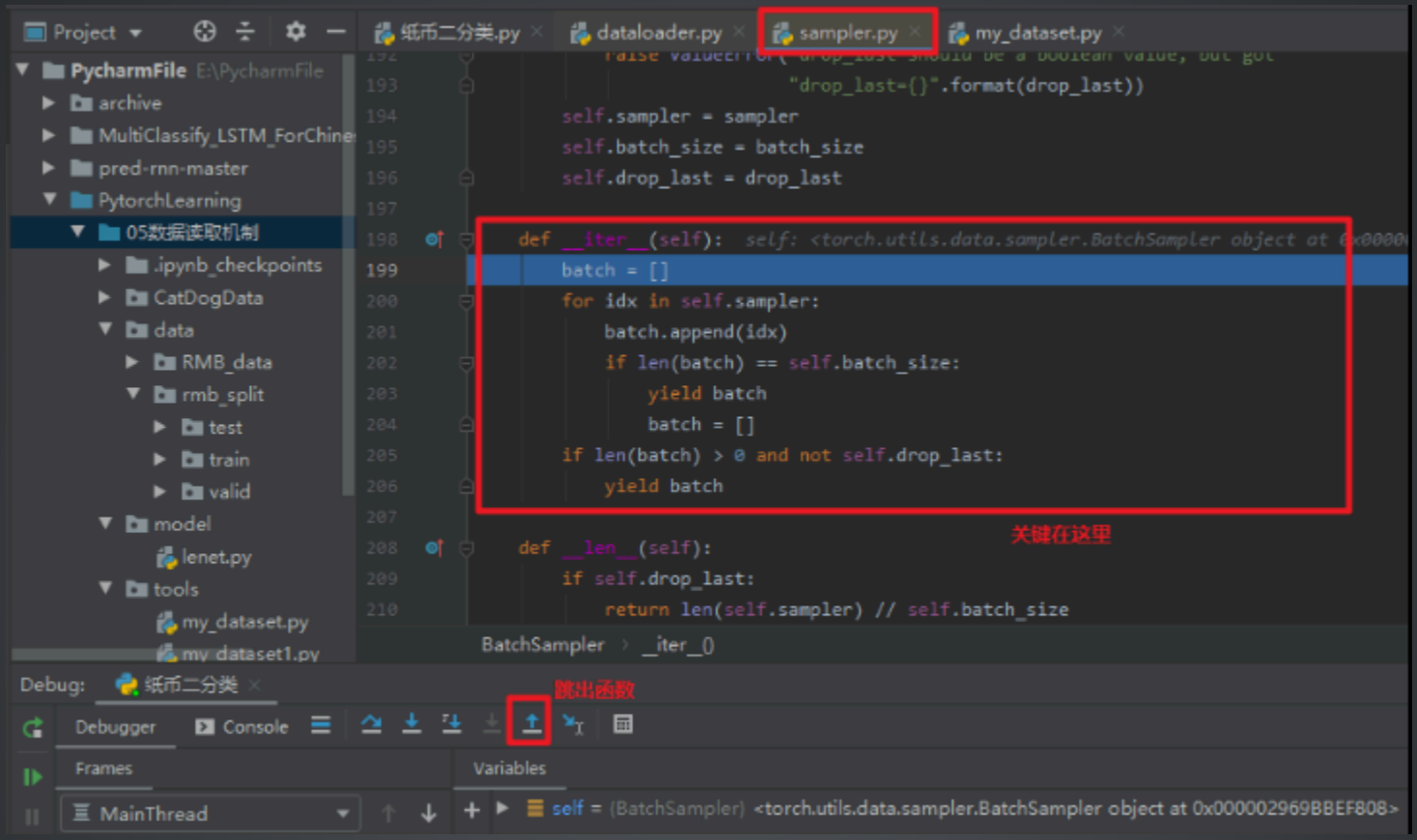
- 上面不是说RMBDataset函数是能返回一个样本和标签吗?
- 这里的这个next，看其字面意义就知道这个是获取下一个样本和标签，重要的两行代码就是红框的那两行，
- `self.__next__index()`获取下一个样本的index，
- 然后`self.dataset_fetcher.fetch(index)`根据index去获取下一个样本， 那么是怎么做到的?

- 继续调试：将光标放到__next__index()这一行，然后点击下面的run to cursor图表，就会跳到这一行，然后stepinto



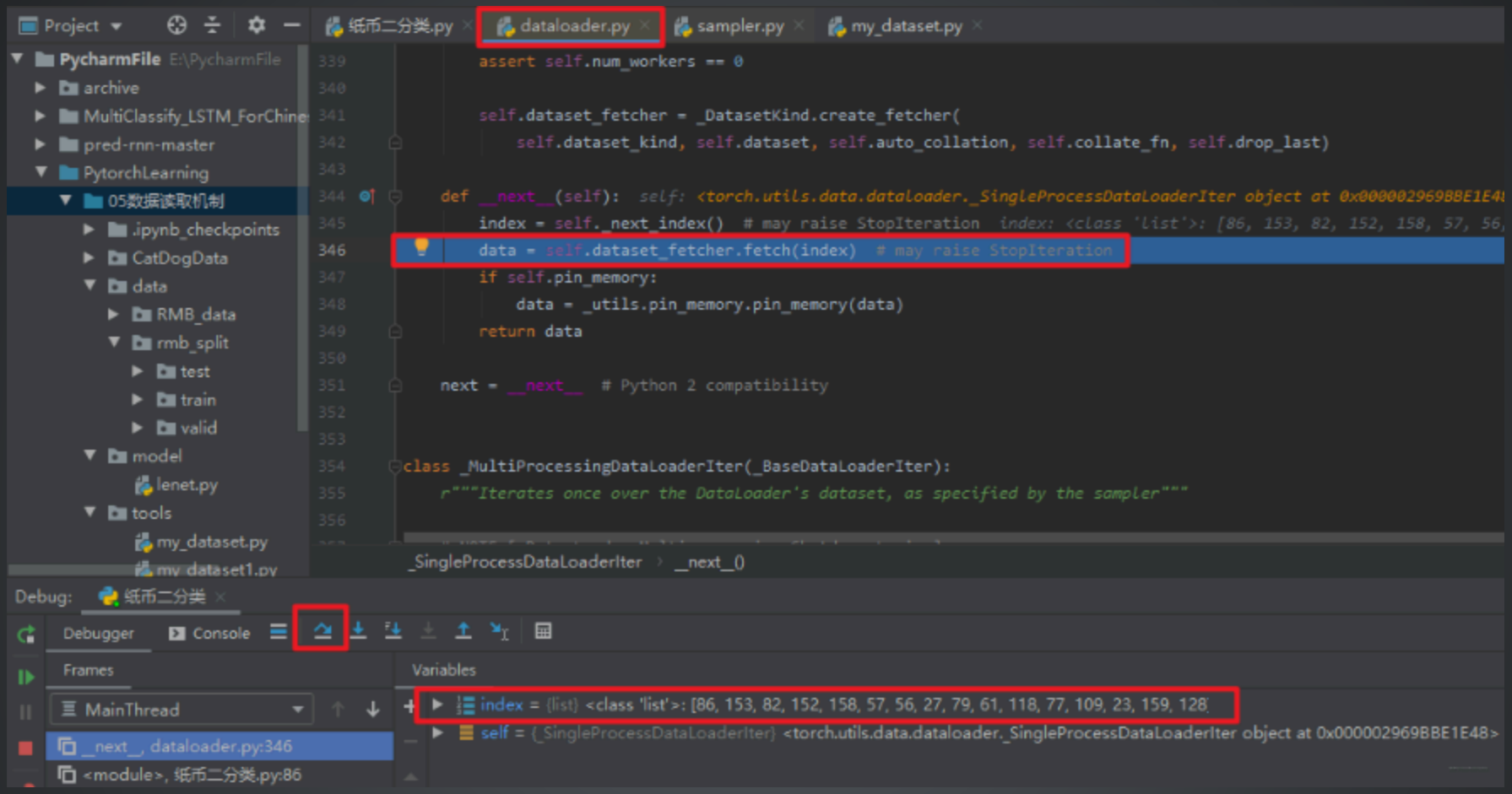
- 发现，这里是返回了一个return next(self.sampler_iter),所以重点应该是这个东西，

我们继续stepinto



- 这里发现进入了sampler.py,这里面重要的就是这个__iter__(self)
- 这个方法正是一次次的去采样我们的数据的索引，然后够了一个batch_size了就返回了。
- 那这一次取到的哪些样本的索引呢?

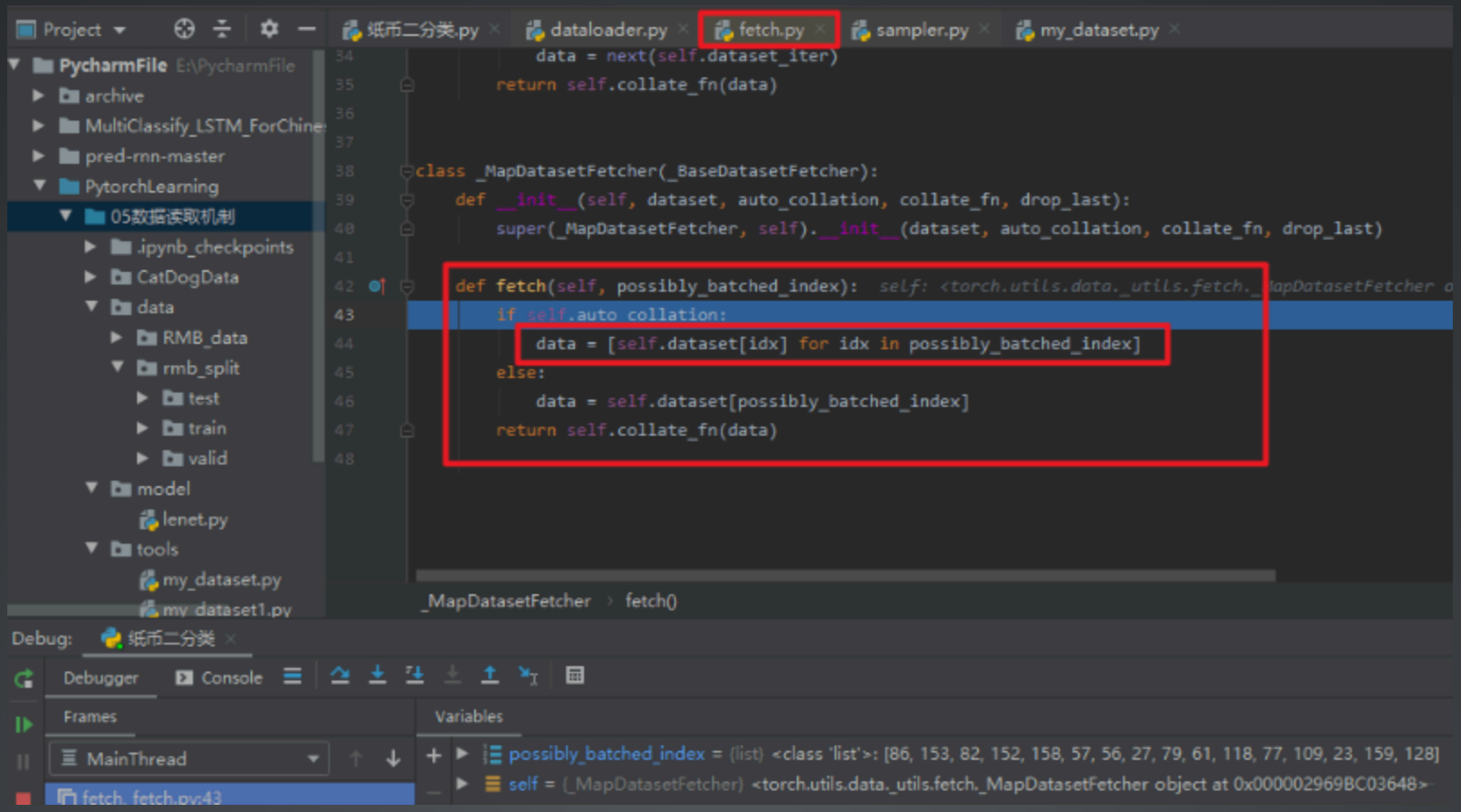
我们可以跳出这个函数，回去看看（连续两次跳出函数，回到dataloader.py）：



- 然后stepover到data这一行
- 这个意思就是说，index这一样代码执行完毕，我们可以看到最下面取到的index（可以和上上张图片，没执行这个函数的时候对比一下）
- 我们的batch_size设置的16， 所以通过上面的sampler.py获得了16个样本的索引。

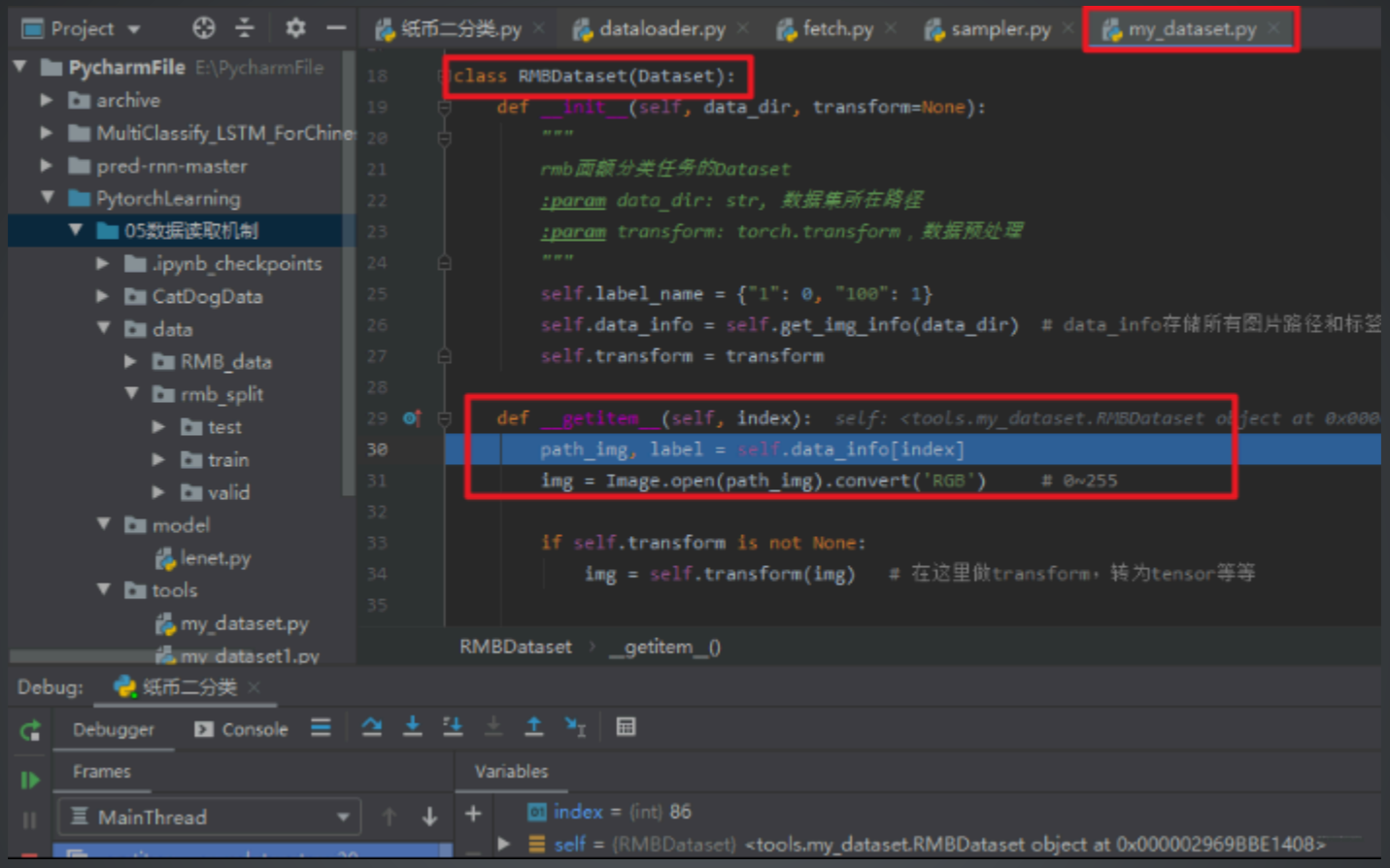
这样，我们就有了一个批次的index，那么就好说了，根据index取不就完事了

- 所以第二行代码data = self.dataset_fetcher.fetch(index)就是取数据去了
- 重点就是这里的dataset_fetcher.fetch方法， 我们继续调试看看它是怎么取数据的。

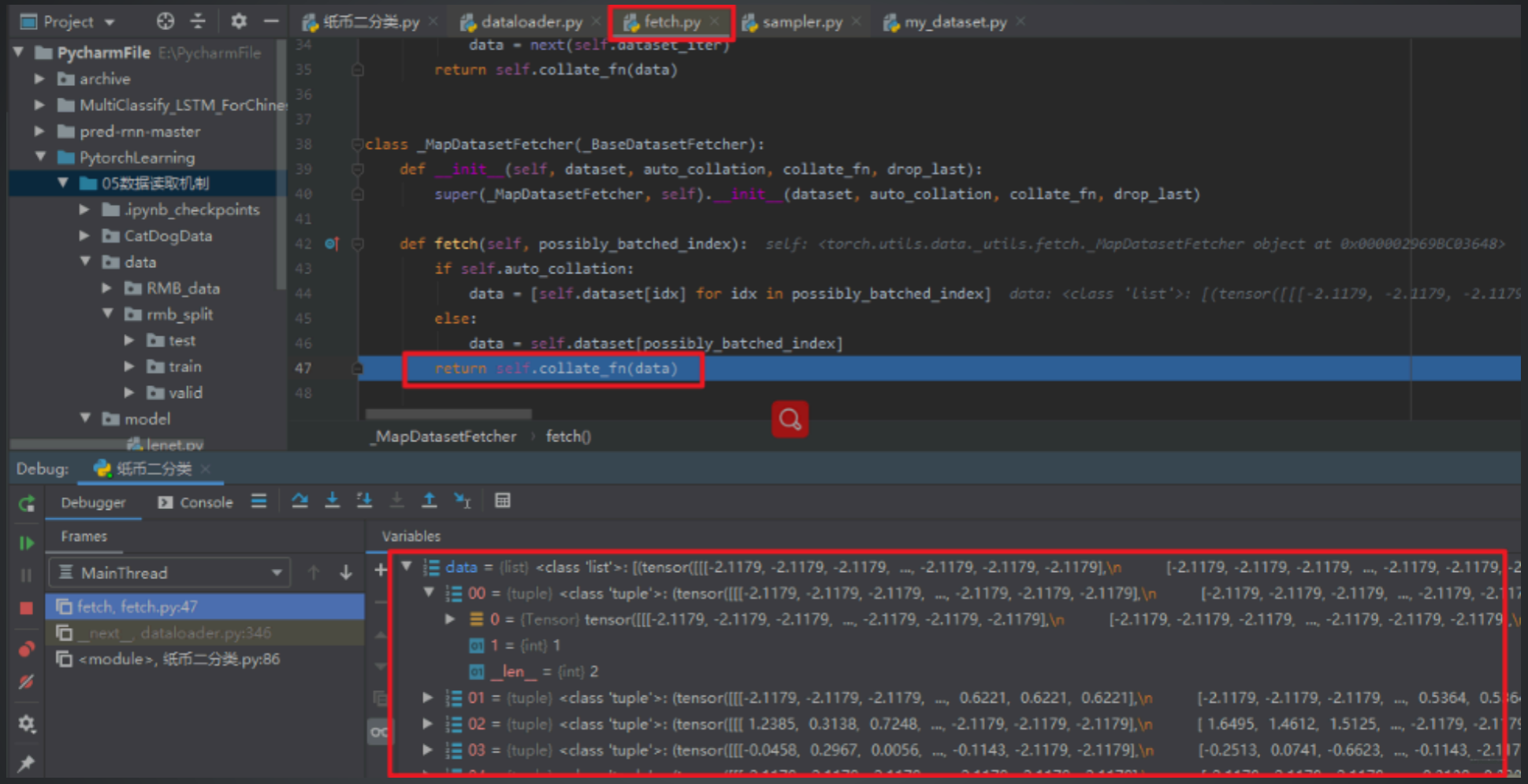


这样进入了fetch.py， 然后核心是这里的fetch方法，这里面会发现调用了`self.dataset[idx]`去获取数据，

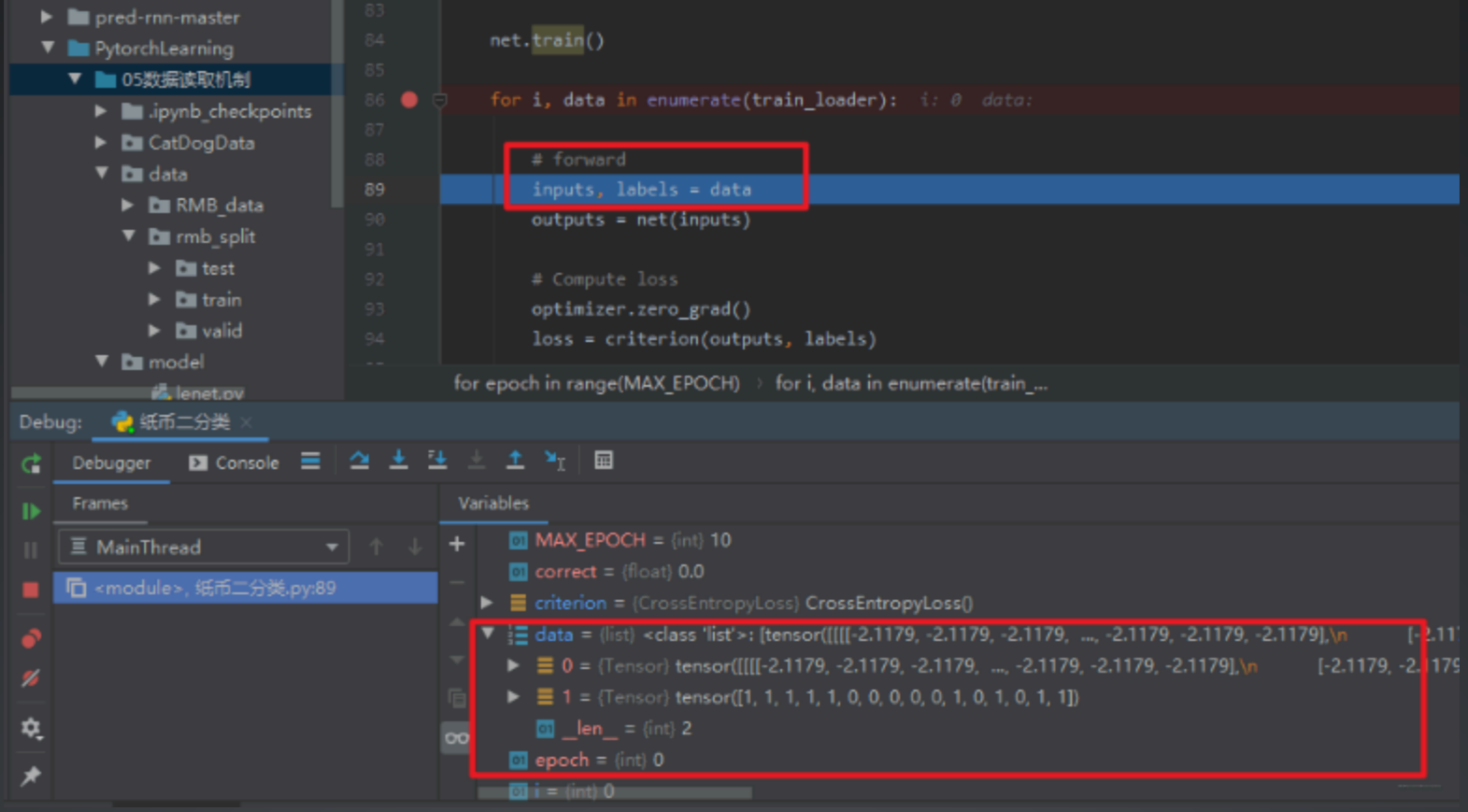
那么我们再步入一步，就看到了重点:



- 会发现，这个方法跳到了我们写的RMBDataset这个类里面
- 调用了__getitem__方法，这个我们知道是获取一个样本的
- 那么就拿到了这个样本的张量和标签。
- 而fetch里面的那个方法是一个列表推导式，所以通过这个方法就能够获取一个batch大小的样本。



取完了一个批次， 然后进入`self.collate_fn(data)`进行整合，就得到了我们一个批次的data，最终我们返回来。



- 就看到了我们第一个批次获得的数据样本了。

$$\frac{\text{samples_num}}{\text{batch_size}}$$

- 所以通过enumerate进行迭代就可以一批批的获取，然后训练模型了。

- 这样所有的批次数据都喂入了模型，就完成了一次epoch。

上面就是DATALOADER读取数据的过程了

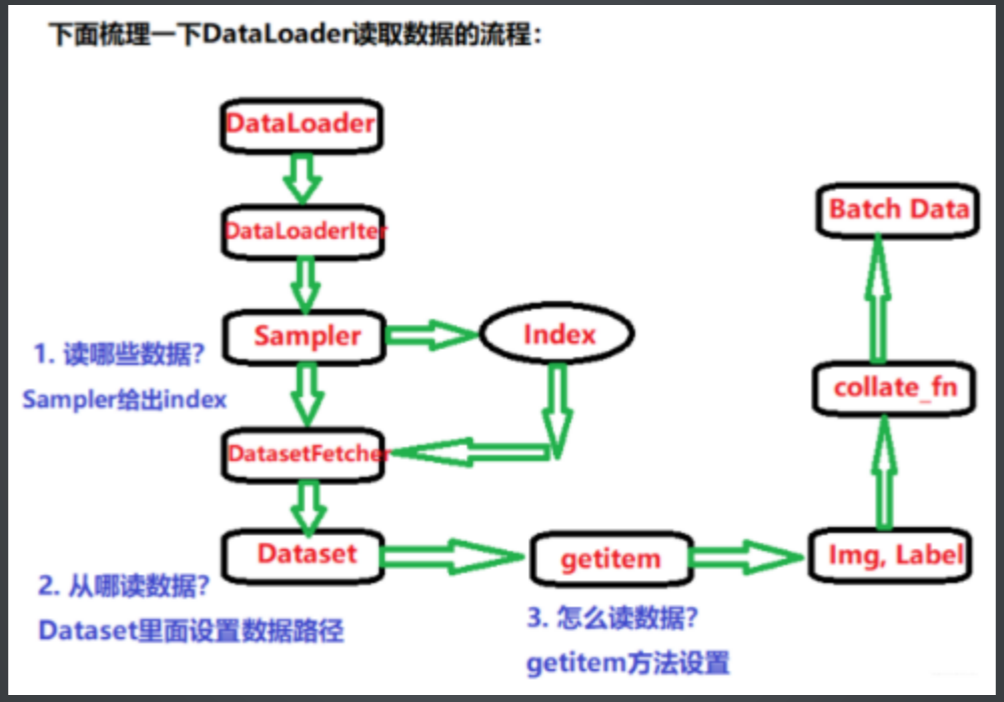
- 所以我们基于那三个问题梳理一遍逻辑，把逻辑关系看懂就好了
- 最后来个流程图再进行梳理。

三个问题：

1. 读哪些数据？ 这个我们是根据Sampler输出的index决定的
2. 从哪读数据？ 这个是Dataset的data_dir设置数据的路径，然后去读
3. 怎么读数据？ 这个是Dataset的getitem方法，可以帮助我们获取一个样本

DataLoader读取数据的过程比较麻烦，用到了四五个.py文件的跳转

梳理这个逻辑关系最好的方式就是流程图：



- 通过这个流程图，把DataLoader读取数据的流程梳理了一遍
- 具体细节不懂没有关系，但是这个逻辑关系应该要把握住
- 把握宏观过程，也能够清晰的看出DataLoader和Dataset的关系。

- 根据前面介绍，DataLoader的作用就是构建一个数据装载器
- 根据我们提供的batch_size的大小， 将数据样本分成一个个的batch去训练模型
- 而这个分的过程中需要把数据取到，这个就是借助Dataset的`getitem`方法。

这样也就清楚了，如果我们想使用Pytorch读取数据的话

- 首先应该自己写一个MyDataset，这个要继承Dataset类并且实现里面的`__getitem__`方法，在这里面告诉机器怎么去读数据。
- 当然这里还有个细节，就是还要覆盖里面的`__len__`方法，这个是告诉机器一共有多少个样本数据。 要不然机器没法去根据`batch_size`的个数去确定有多少批数据。

这个写起来也很简单，返回总的样本的个数即可。

```
def __len__(self):
        return len(self.data_info)
```

- 这样， 机器就可以根据Dataset去硬盘中读取数据
- 接下来就是用DataLoader构建一个可迭代的数据装载器
- 传入如何读取数据的机制Dataset
- 传入batch_size, 就可以返回一批批的数据了。

当然这个装载器具体使用是在模型训练的时候。 当然，由于DataLoader是一个可迭代对象，当我们构建完毕之后，也可以简单的看下里面的数据到底长什么样， 大致代码是：

```
for x, y in train_loader:
    print(x, y)
    break

# 这样应该能看到一个批次的数据
```

好了，上面就是Pytorch读取机制DataLoader和Dataset的原理部分了。

人民币二分类的数据模块里面，除了数据读取机制DataLoader，还涉及了一个图像的预处理模块transforms， 是对图像进行预处理的

下面我们再看看这个的原理

==3.3-3. PYTORCH的图像预处理TRANSFORMS==

- transforms是常用的图像预处理方法
- 这个在torchvision计算机视觉工具包中，
- 在安装Pytorch的时候应该顺便安装这个torchvision

在torchvision中，有三个主要的模块：

- `torchvision.transforms`: 常用的图像预处理方法, 比如标准化，中心化，旋转，翻转等操作
- `trochvision.datasets`: 常用的数据集的dataset实现， MNIST, CIFAR-10, ImageNet等
- `torchvision.models`: 常用的模型预训练, AlexNet, VGG, ResNet, GoogLeNet等。

我们这次看图像预处理模块transforms， 主要包括下面的方法：

数据中心化，数据标准化，缩放，裁剪，旋转，翻转，填充，噪声添加，灰度变换，线性变换，仿射变换，亮度、饱和度及对比度变换。

3.3-3.1 看看二分类任务中用到的TRANSFORMS的方法

下面我们可以看看人民币二分类任务中用到的图像预处理的方法了：
导入：

```
import torchvision.transforms as transforms
```

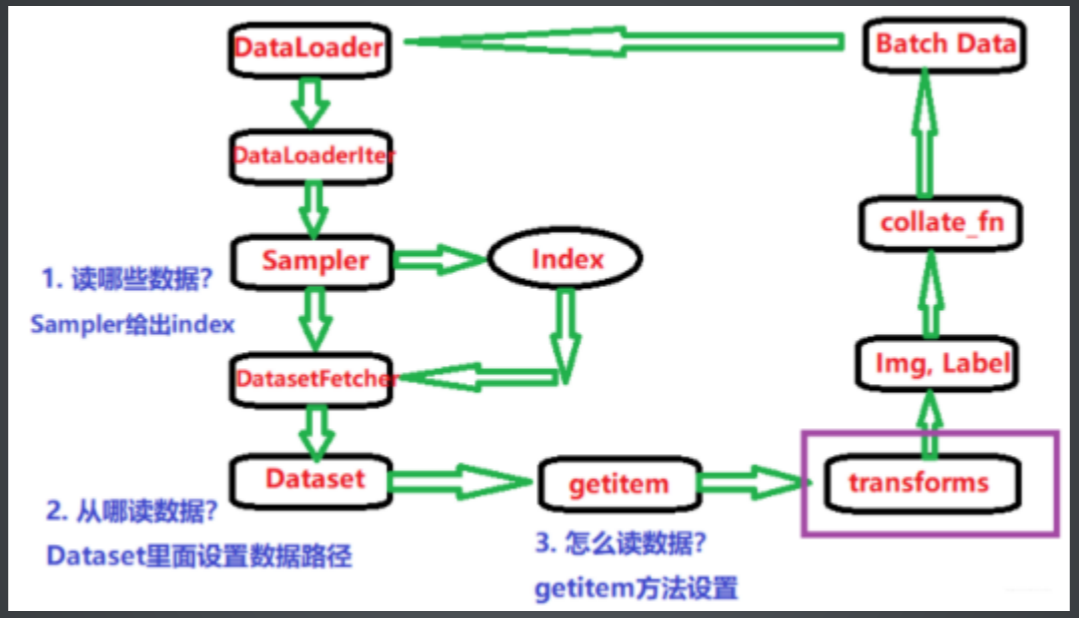
```
train_transform = transforms.Compose([  train_transform: Compose(
    transforms.Resize((32, 32)),
    transforms.RandomCrop(32, padding=4),
    transforms.ToTensor(),
    transforms.Normalize(norm_mean, norm_std),
])

valid_transform = transforms.Compose([  valid_transform: Compose(
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize(norm_mean, norm_std),
])
```

- transforms.Compose方法是将一系列的transforms方法进行有序的组合包装，具体实现的时候，依次的用包装的方法对图像进行操作。
- transforms.Resize方法改变图像大小
- transforms.RandomCrop方法对图像进行裁剪（这个在训练集里面用，验证集就用不到了）
- transforms.ToTensor方法是将图像转换成张量，同时会进行归一化的一个操作，将张量的值从0-255转到0-1
- transforms.Normalize方法是将数据进行标准化

- transforms这个函数是在RMBDataset的__getitem__方法中调用的。也就是在这里处理的图像。 ```python def **getitem**(self, index): path_img, label = self.data_info[index] img = Image.open(path_img).convert('RGB') # 0~255 if self.transform is not None: img = self.transform(img) # 在这里做transform，转为tensor等等 return img, label

```
```

但是逻辑关系依然要知道：



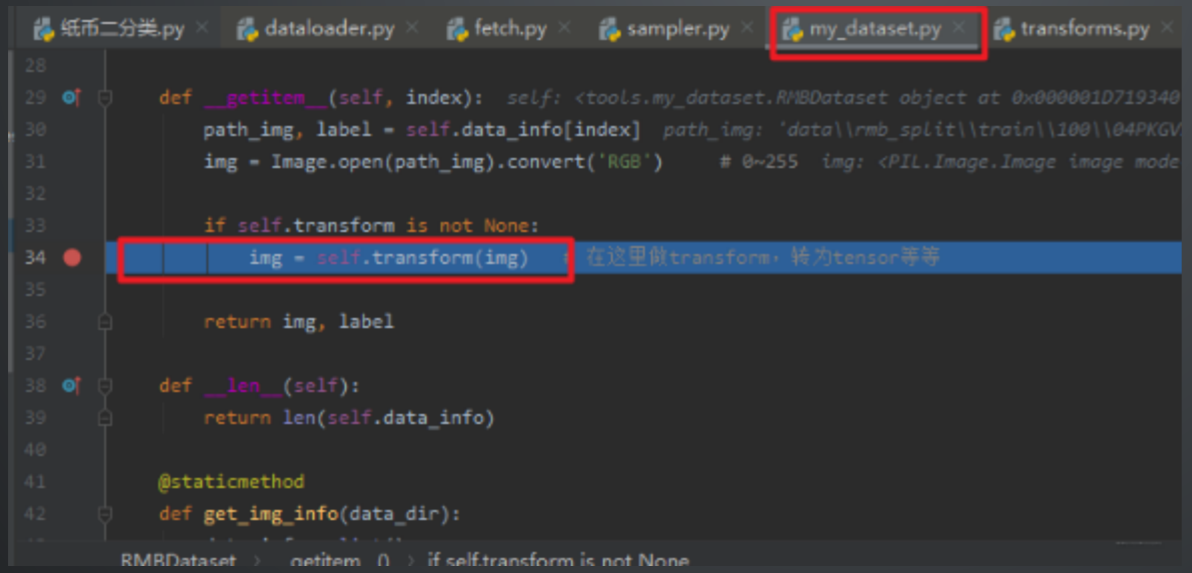
了解了图像处理的transforms机制，我们下面学习一个比较常用的数据预处理机制，叫做数据标准化：

- transforms.Normalize: 逐channel的对图像进行标准化。

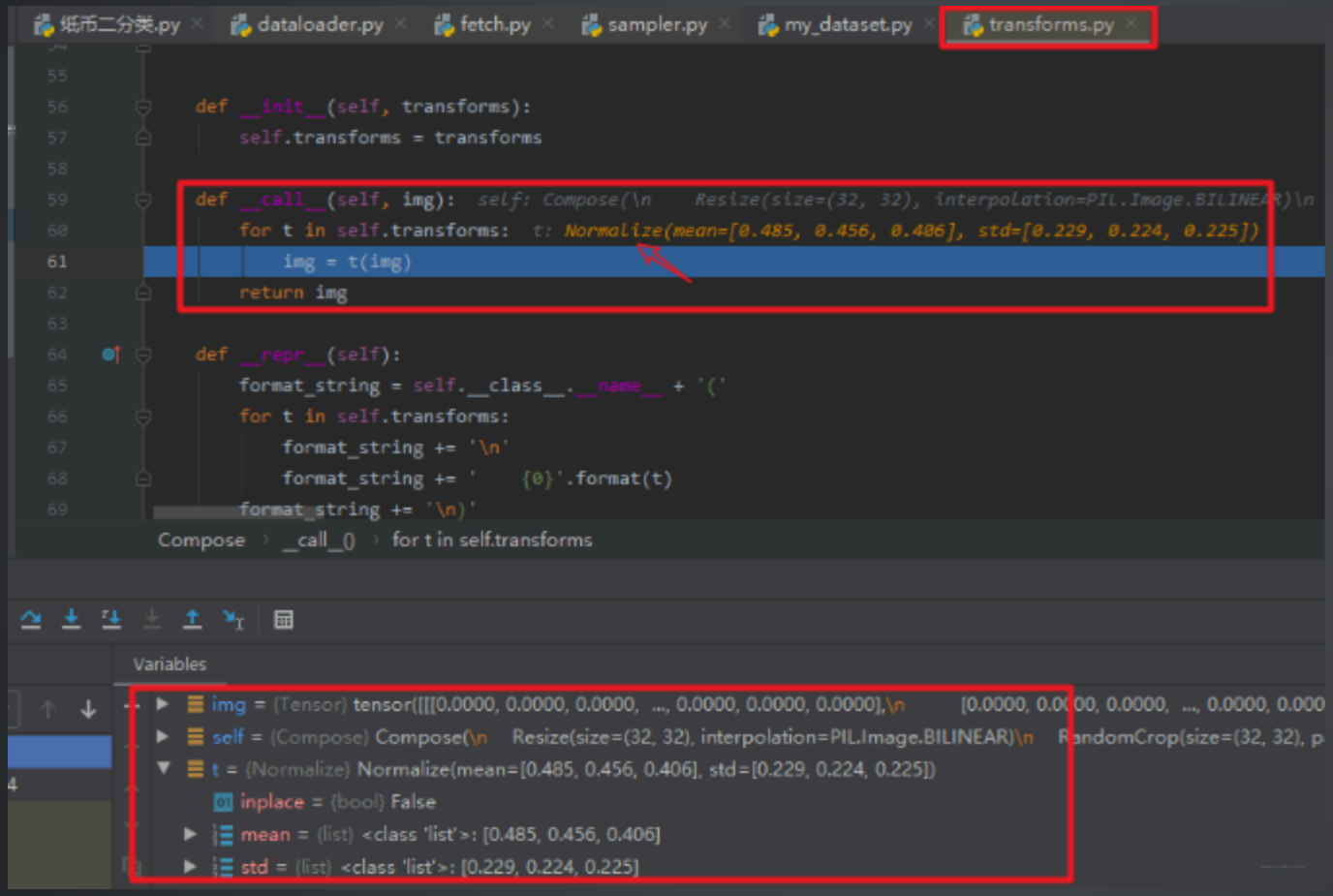
$$output = (input - mean)/std$$

```
transforms.Normalize(mean,
                     std,
                     inplace=False)
```

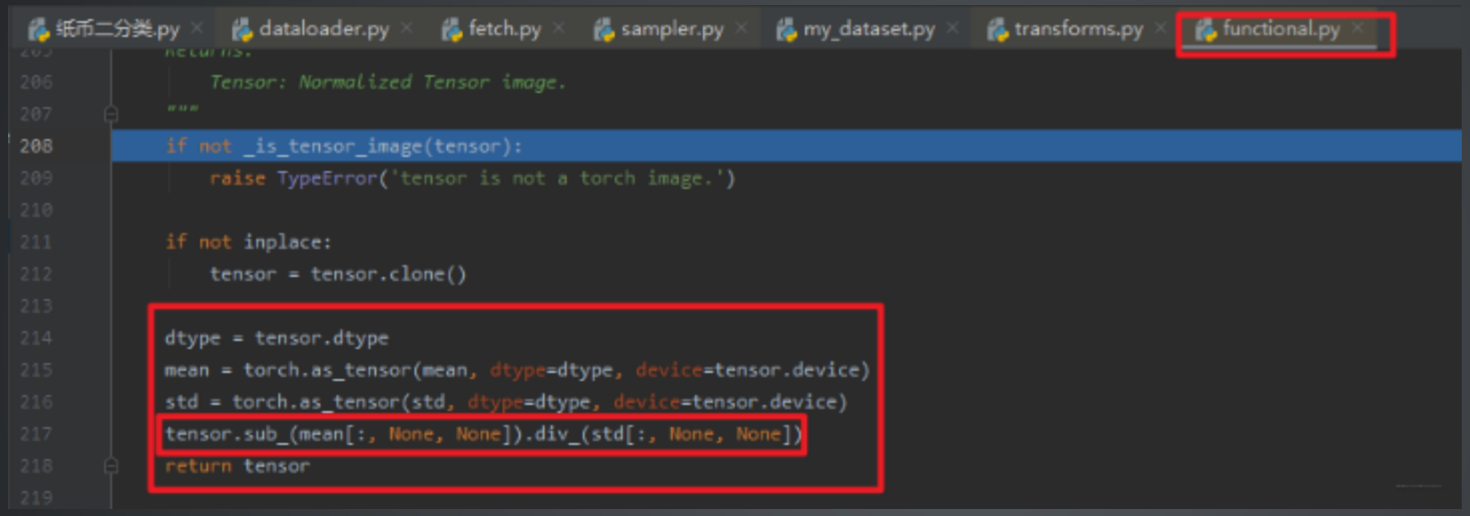
- 这个参数再进行调试一下，看看是怎么变的：
- 依然是先打断点，然后步入这个函数：



我们进入了transforms.py， 这里面的__call__里面就是那一系列的数据处理方法



然后点几次stepover就到了Normalize这个操作，这时候我们再次步入，到了Normalize类，这里面有一个call函数调用了pytorch库里面的Normalize函数， 我们再次步入：



这里就有图有真相了。

- Normalize的处理作用就是有利于加快模型的收敛速度。

提问：在图像预处理模块中，`transforms.Normalize()` 的作用是什么？

- A. 随机裁剪图像
- B. 将图像转换为张量
- C. 对图像进行标准化处理
- D. 调整图像的亮度和对比度

正确答案: C

答案讲评: `transforms.Normalize()` 用于对图像进行标准化处理，它将图像的每个通道按给定的均值和标准差进行归一化。这有助于加快模型的收敛速度

3.3-3.2 TRANSFORMS的其他图像增强方法

- 数据增强
  - 是对**训练集**进行变换，使训练集更丰富，从而让模型更具**泛化能力**
- 图像裁剪
  - `transforms.CenterCrop(size)`: 图像中心裁剪图片, size是所需裁剪的图片尺寸，如果比原始图像大了， 会默认填充0。
  - `transforms.RandomCrop(size, padding=None, pad_if_needed=False, fill=0, padding_mode='constant)`: 从图片中位置随机裁剪出尺寸为size的图片， size是尺寸大小，padding设置填充大小
  - `transforms.RandomResizedCrop(size, scale=(0.08, 1.0), ratio=(3/4, 4/3), interpolation)`: 随机大小，长宽比裁剪图片。 scale表示随机裁剪面积比例，ratio随机长宽比， interpolation表示插值方法。

3. 图像的翻转和旋转

(1) RandomHorizontalFlip(p=0.5)，RandomVerticalFlip(p=0.5):依概率水平或者垂直翻转图片， p表示翻转概率

(2) RandomRotation(degrees，resample=False，expand=False，center=None):随机旋转图片， degrees表示旋转角度， resample表示重采样方法，expand表示是否扩大图片，以保持原图信息。

4. 图像变换

- `transforms.Pad(padding, fill=0, padding_mode='constant')`:对图片边缘进行填充
- `transforms.ColorJitter(brightness=0, contrast=0, saturation=0, hue=0)`:调整亮度、对比度、饱和度和色相， 这个是比较实用的方法， brightness是亮度调节因子， contrast对比度参数， saturation饱和度参数， hue是色相因子。

3.3-3.3 TRANSFORMS的选择操作

对几个transforms的操作进行选择，使得图像预处理更加的灵活。

1. `transforms.RandomChoice([transforms1, transforms2, transforms3])`: 从一系列transforms方法中随机选一个
2. `transforms.RandomApply([transforms1, transforms2, transforms3], p=0.5)`: 依据概率执行一组transforms操作
3. `transforms.RandomOrder([transforms1, transforms2, transforms3])`: 对一组transforms操作打乱顺序

关于Pytorch的transforms操作基本上就搞定， 上面只是整理了一些常用的函数，如果真的需要，具体细节还得去看官方文档。

虽然Pytorch提供了很多的transforms方法， 但是在实际工作中，可能需要自己的项目去自定义一些transforms方法

3.3-3.4 自定义TRANSFORMS

我们上面的代码调试中看到了在Compose这个类里面调用了一系列的transforms方法， 还记得这个吗？ 我们再回顾一遍这个运行机制：

```
class Compose(object):
    def __call__(self, img):
        for t in self.transforms:
            img = t(img)
        return img
```

- 我们对Compose里面的这些transforms方法执行一个for循环
- 每次挑取一个方法进行执行。 也就是**transforms方法仅接收一个参数，返回一个参数**
- 然后就是for循环中，**上一个transforms的输出正好是下一个transforms的输入，所以数据类型要注意匹配**
- 这就是自定义transforms的两个要素。

具体需要什么transform操作，这里在主程序中预先设置了

01_PyTorch_Practice > bigAI-03-03-20241118.py > ...

```python
# ============================ step 1/5 数据 ============================

train_dir = os.path.join(rmb_split_dir, "train")
valid_dir = os.path.join(rmb_split_dir, "valid")

norm_mean = [0.485, 0.456, 0.406]
norm_std = [0.229, 0.224, 0.225]

train_transform = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.RandomCrop(32, padding=4),
    transforms.RandomGrayscale(p=0.9),
    transforms.ToTensor(),
    transforms.Normalize(norm_mean, norm_std),
])


valid_transform = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize(norm_mean, norm_std),
])

```

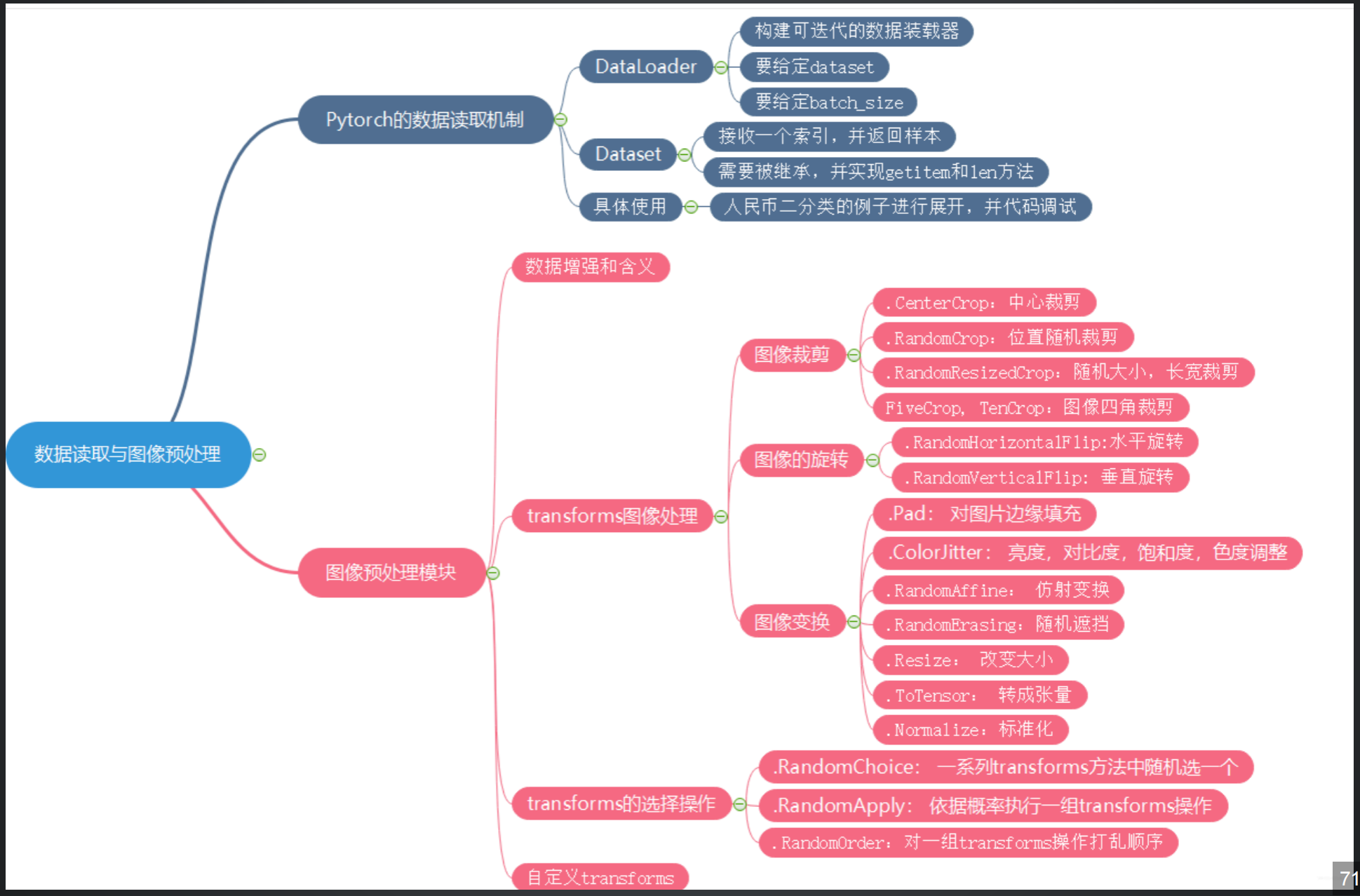
数据增强策略原则： **让训练集与测试集更接近**。

- 空间位置上： 可以选择平移
- 色彩上： 灰度图，色彩抖动
- 形状： 仿射变换
- 上下文场景： 遮挡，填充

4 总结梳理

- 首先是整理了Pytorch的数据读取机制， 学习到了两个数据读取的关键DataLoader和Dataset
- 通过一个人民币二分类的例子具体看了下这两个是如何使用的，以及它们之间的关系和原理
- 这个是通过debug进行描述的，debug的这种操作本身也非常重要，便于阅读理解源码
- 然后又学习了Pytorch的图像处理模块transforms， 这一模块主要是整理了各种图像处理的方法

思维导图


数据读取与图像预处理
Pytorch的数据读取机制
DataLoader
构建可迭代的数据装载器
要给定dataset
要给定batch_size
Dataset
接收一个索引，并返回样本
需要被继承，并实现getitem和len方法
具体使用
人民币二分类的例子进行展开，并代码调试
图像预处理模块
数据增强和含义
transforms图像处理
图像裁剪
.CenterCrop：中心裁剪
.RandomCrop：位置随机裁剪
.RandomResizedCrop：随机大小，长宽裁剪
FiveCrop，TenCrop：图像四角裁剪
图像的旋转
.RandomHorizontalFlip:水平旋转
.RandomVerticalFlip：垂直旋转
图像变换
.Pad：对图片边缘填充
.ColorJitter：亮度，对比度，饱和度，色度调整
.RandomAffine：仿射变换
.RandomErasing：随机遮挡
.Resize：改变大小
.ToTensor：转成张量
.Normalize：标准化
transforms的选择操作
.RandomChoice：一系列transforms方法中随机选一个
.RandomApply：依据概率执行一组transforms操作
.RandomOrder：对一组transforms操作打乱顺序
自定义transforms

关于Pytorch的数据模块，到这里就基本结束 学习的逻辑就是按照机器学习的那五大步骤进行

- 数据模块
- 模型模块
- 损失函数
- 优化器
- 训练

下一次我们进入模型模块，看看模型模块的具体细节 😉